Containerizing a Web Application with Docker

A Complete Step-by-Step Tutorial

Learn how to package your HTML web app into a portable Docker container — from zero to running in a browser.

Table of Contents

1. Introduction & Overview

Docker is an open-source platform that enables developers to package applications and all their dependencies into a single, portable unit called a container. Containers run consistently on any machine — whether it is a developer laptop, a CI/CD server, or a cloud virtual machine — eliminating the classic "it works on my machine" problem.

In this tutorial you will containerize a simple HTML/CSS/JS web application served by an Nginx web server. By the end you will be able to:

- Build a Docker image from a Dockerfile.
- Run the image as a container and expose it on a local port.
- Access the running web app in a browser.
- Stop, remove, and optionally publish the image.

> 📌 **Note:** No prior Docker experience is needed. Every command is explained in detail.

2. Understanding the Project Files

2.1 index.html — The Web Application

index.html is the entry point of the web application. It contains the HTML markup, inline CSS styling, and any JavaScript that renders the user interface in the browser. Nginx will serve this file when a visitor navigates to the container's exposed port.

2.2 Dockerfile — The Build Recipe

The Dockerfile is a plain-text script that tells Docker exactly how to assemble the image. It defines the base operating system image, copies application files into the image, and specifies which port to expose and which command to run at start-up.

```
# ── Dockerfile ──────────────────────────────────
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

3. Prerequisites & Installation

Before you begin, make sure the following software is installed on your machine.

3.1 Install Docker

Operating System	Installation Method
Windows 10 / 11	Download Docker Desktop from https://docs.docker.com/desktop/install/windows-install/ and run the installer.
macOS	Download Docker Desktop from https://docs.docker.com/desktop/install/mac-install/ and drag to Applications.
Ubuntu / Debian Linux	Run: sudo apt-get update && sudo apt-get install docker.io -y

3.2 Verify Docker Installation

Open a terminal (Command Prompt / PowerShell on Windows, Terminal on Mac/Linux) and run:

```
docker --version

# Expected output (version numbers may differ):
Docker version 26.1.3, build b72abbb
```

⚠ **Warning:** If the command is not found, restart your terminal or machine after installation.

4. Project Directory Structure

Keep both files in the same folder. Docker's build context is the directory you pass to the docker build command, and the COPY instruction in the Dockerfile looks for files relative to that context.

```
my-web-app/
├── Dockerfile
└── index.html
```

Tip: The folder name does not matter. What matters is that Dockerfile and index.html are in the SAME directory.

5. Understanding the Dockerfile — Line by Line

5.x Base Image

```
FROM nginx:alpine
```

Pulls the official Nginx image built on Alpine Linux (a tiny ~5 MB OS). Nginx is a production-grade web server that will serve our HTML file.

5.x Copy Application File

```
COPY index.html /usr/share/nginx/html/index.html
```

Copies index.html from the build context (your local folder) into the container's file system at the location Nginx expects to find web files.

5.x Declare Port

```
EXPOSE 80
```

Documents that the container listens on port 80 (HTTP). This is metadata for developers and orchestration tools — actual port mapping is done at runtime.

5.x Start Command

```
CMD ["nginx", "-g", "daemon off;"]
```

Starts Nginx in the foreground when the container launches. "daemon off" is required so Docker can track the process and keep the container alive.

6. Step 1 – Build the Docker Image

Building the image reads your Dockerfile, executes each instruction in order, and saves the result as a layered image on your local machine.

6.1 Navigate to the Project Folder

```
# On Windows (PowerShell)
cd C:\Users\YourName\my-web-app

# On macOS / Linux
cd ~/my-web-app
```

6.2 Run the Build Command

```
docker build -t my-web-app:v1 .
```

Breaking down each part of the command:

- docker build — tells Docker to build a new image.
- -t my-web-app:v1 — tags the image with the name "my-web-app" and version "v1".
- . (dot) — sets the build context to the current directory.

6.3 Expected Build Output

```
[+] Building 3.2s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => [internal] load .dockerignore
 => [internal] load metadata for docker.io/library/nginx:alpine
 => [1/2] FROM docker.io/library/nginx:alpine
 => [2/2] COPY index.html /usr/share/nginx/html/index.html
 => exporting to image
 => => naming to docker.io/library/my-web-app:v1
```

Tip: The first build downloads the nginx:alpine image (~10 MB). Subsequent builds use the cached layer and complete in under a second.

7. Step 2 – Verify the Image Was Created

```
docker images
```

You should see an entry similar to:

```
REPOSITORY     TAG    IMAGE ID       CREATED         SIZE
my-web-app     v1     d3f1a2b4c5e6   2 minutes ago   43.2MB
nginx          alpine 9f8e7d6c5b4a   2 weeks ago     40.7MB
```

> **Tip:** The image is now stored locally. You can share it by pushing to Docker Hub (Step 7).

8. Step 3 – Run the Container

A container is a running instance of an image. The command below starts the container and maps a port on your host machine to port 80 inside the container.

```
docker run -d -p 8080:80 --name webapp my-web-app:v1
```

Flag breakdown:

- -d — Detached mode: runs in the background, freeing the terminal.
- -p 8080:80 — Maps host port 8080 → container port 80.
- --name webapp — Assigns a human-friendly name to the container.
- my-web-app:v1 — The image to instantiate.

8.1 Expected Output

```
3f7a1b2c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f
# (a full container ID hash — yours will differ)
```

9. Step 4 – Access the Web Application in a Browser

Open any web browser and navigate to:

```
http://localhost:8080
```

You should see your index.html page rendered in the browser. The request travels: Browser → Host port 8080 → Docker networking → Container port 80 → Nginx → index.html.

> **Remote Access:** If you are running Docker inside a virtual machine or on a remote server, replace "localhost" with the VM's or server's IP address.

10. Step 5 – Inspect the Running Container

10.1 List Running Containers

```
docker ps
```

```
CONTAINER ID   IMAGE           COMMAND                  CREATED
STATUS         PORTS                  NAMES
3f7a1b2c4d5e   my-web-app:v1   "/docker-entrypoint.…"   3 minutes ago   Up 3
minutes  0.0.0.0:8080->80/tcp   webapp
```

10.2 View Container Logs

```
docker logs webapp
```

10.3 Open a Shell Inside the Container

```
docker exec -it webapp sh

# Inside the container:
ls /usr/share/nginx/html/
# You should see: index.html
exit
```

11. Step 6 – Stop & Remove the Container

11.1 Stop the Running Container

```
docker stop webapp
```

11.2 Remove the Container

```
docker rm webapp
```

11.3 (Optional) Remove the Image

```
docker rmi my-web-app:v1
```

> ⚠ **Note:** Stopping a container does NOT delete it. You still need "docker rm" to free disk space.

12. Step 7 – Push the Image to Docker Hub (Optional)

Docker Hub is a public (or private) registry where you can store and share images. Follow these steps to publish your image.

12.1 Create a free account

```
Go to https://hub.docker.com and sign up.
```

12.2 Log in from the terminal

```
docker login
# Enter your Docker Hub username and password when prompted.
```

12.3 Tag the image with your username

```
docker tag my-web-app:v1 <your-dockerhub-username>/my-web-app:v1
```

12.4 Push the image

```
docker push <your-dockerhub-username>/my-web-app:v1
```

12.5 Pull on any other machine

```
docker pull <your-dockerhub-username>/my-web-app:v1
```

13. Troubleshooting Common Issues

Problem	Solution
Port already in use	Change the host port: -p 8081:80 instead of -p 8080:80, or stop the conflicting process.
"Cannot connect to the Docker daemon"	Ensure Docker Desktop is running, or start the daemon: sudo systemctl start docker
COPY failed: file not found	Make sure index.html is in the SAME directory as the Dockerfile when you run docker build.
Browser shows "Connection Refused"	Confirm the container is running with docker ps. Check you are using the correct host port.
Image not found when running	Run docker images to verify the tag name matches exactly, including case sensitivity.

14. Quick-Reference Cheat Sheet

```
# Build image
docker build -t my-web-app:v1 .

# List local images
docker images

# Run container (detached, port mapped)
docker run -d -p 8080:80 --name webapp my-web-app:v1

# List running containers
docker ps

# View logs
docker logs webapp

# Open shell inside container
docker exec -it webapp sh

# Stop container
docker stop webapp

# Remove container
docker rm webapp

# Remove image
docker rmi my-web-app:v1

# Push to Docker Hub
docker push <username>/my-web-app:v1
```

15. Summary

In this tutorial you learned the complete workflow for containerizing a web application with Docker:

1. Wrote a Dockerfile using nginx:alpine as the base image.
2. Placed index.html alongside the Dockerfile in one project folder.
3. Built a Docker image with docker build -t my-web-app:v1 .
4. Verified the image appeared in docker images.
5. Launched a container with docker run -d -p 8080:80 --name webapp my-web-app:v1.
6. Opened http://localhost:8080 in a browser to view the live application.
7. Inspected logs and the container internals with docker logs and docker exec.
8. Stopped and removed the container cleanly.
9. (Optionally) pushed the image to Docker Hub for public distribution.

🚀 **Next Steps:** You can extend this setup by adding a .dockerignore file, using multi-stage builds for compiled apps, or orchestrating multiple containers with Docker Compose.